

An Informal Fleet: Operating a Leaderless Workflow Runtime on Commodity Desktops — an Experience Report

Abstract

1. Introduction
2. The fleet and its management model
3. The intensive window, quantified
4. The incident catalog
 - 4.1 Sleep/wake and the zombie runners
 - 4.2 DHCP drift and identity
 - 4.3 Image-layer disk exhaustion (and the 113 GB mini)
 - 4.4 Emulation boot storms
 - 4.5 Wedged container runtimes and stuck agents
 - 4.6 What never happened
5. What earned its keep
6. Limits, honestly
7. Conclusion

An Informal Fleet: Operating a Leaderless Workflow Runtime on Commodity Desktops — an Experience Report

Research companion to the Facetwork thesis ([thesis.md](#)). Empirical basis: operating records of a four-host fleet (July 2026 intensive window: nine image rollouts and ~15 production workflow campaigns in 72 hours), the fleet tooling in [scripts/lib/fleet/](#), and the incident record in project memory and [docs/architecture/lessons-learned.md](#).

Abstract

Workflow-engine literature assumes data-center conditions: stable hosts, homogeneous provisioning, an orchestration substrate (Kubernetes), and an operations team. We report on eighteen months of operating the opposite: a leaderless workflow fleet on four consumer machines — two Apple-silicon desktops and two low-power minis running x86 containers under emulation — coordinated only by a shared MongoDB and object store, administered by one person, with runner hosts free to sleep, reboot, change IP, and fill their disks. We describe the design properties that made this survivable (statelessness of runners, atomic queue claims, derived routing, self-healing registration), quantify an intensive 72-hour window in which the fleet absorbed nine full image rollouts while shipping production work, and catalog the incident classes that consumer hardware contributes: sleep/wake heartbeat false-positives, DHCP identity drift, image-layer disk exhaustion, emulation-inflated boot storms, and wedged container runtimes. For each we give the failure signature, the recovery that worked, and the permanent mitigation now in the platform. Our conclusion is unfashionable: for small teams and research groups, the machines already on desks are a viable execution tier — *if* the runtime is designed so that every runner is disposable and every piece of authority lives in the data plane.

1. Introduction

The question this report answers is practical: what actually happens when you run a distributed workflow engine on the computers a small team already owns? Not a simulated degraded environment — the real thing: machines that sleep when idle, renew DHCP leases, run Docker Desktop rather than containerd on bare metal, emulate foreign architectures, and share their disks with a person's photo library.

Facetwork's architecture makes an unusual bet that renders the question answerable. There is no control plane to keep alive: infra services (MongoDB, MinIO, a dashboard, an image registry) are URL-addressed processes on one stable host, and *every other machine is a stateless, leaderless runner*. Runners claim work via atomic single-document updates; task routing keys are derived from intrinsic facts about the work rather than configured; durable inputs and outputs live in the object store, so no runner's disk matters. The fleet controller is a per-host reconciliation agent (`fleet-agent`) polling a versioned config document — closer to a package manager than to a scheduler.

§2 describes the fleet and its management model. §3 quantifies the intensive window. §4 is the incident catalog — the report's core. §5 distills the design properties that earned their keep and the ones we had to add. §6 states the limits honestly.

2. The fleet and its management model

Four hosts on a home-office LAN:

Host	Class	Role	Notes
server3	Apple-silicon desktop, 14 TB disk	infra + heavy runner tier	MongoDB, MinIO, dashboard, image registry; never sleeps
MaxPro	Apple-silicon desktop	heavy runner tier	OSM imports, wide fan-outs; may sleep
server1	mini, 113 GB disk	light runner tier	x86 containers under emulation
server2	mini	light runner tier	x86 containers under emulation

Roughly 55–70 runner containers fleet-wide (one per data domain per host, plus scaled tiers), all built from one image. Management primitives:

- **Central config, local reconciliation.** `fleet_config` is one versioned Mongo document (image tag, per-role replicas, server-group gating). Each host's `fleet-agent` (a `launchd` daemon) polls it and reconciles its own containers — pull, recreate, scale. There is nothing to SSH into during normal operation; `fw fleet set --image <tag>` is the whole deploy interface.
- **Identity by stable name.** A server catalog maps stable names to current addresses, resolved at startup and reconcile — because on DHCP, addresses are weather.
- **Capability tiers, not scheduling.** Server groups gate which *roles* start on which hosts (heavy OSM work never lands on an emulated mini); within a role, placement is whoever claims first. There is no bin-packing and no scheduler to operate.
- **Rollout recipe.** Build on the infra host, push to the local registry, serially pre-pull on each host (staggering a ~2 GB transfer that would otherwise arrive at every host simultaneously), flip the config, let agents converge. Total operator involvement: one command and ~25 minutes of unattended convergence.

3. The intensive window, quantified

A 72-hour engineering push (runtime hardening plus a new language feature) exercised the fleet far beyond normal duty:

- **Nine full image rollouts** (config v83 → v91), each recreating every runner container on every host. All nine converged; none required rollback; two required one manual nudge each (a stuck agent kickstart, a disk-full recovery — §4).
- **Production throughput continued throughout:** four continental-scale workflow campaigns (12–13k steps, 3–10k tasks each) ran to completion *between and during* rollouts, plus ~10 smaller verification workflows.
- **Convergence envelope:** config flip to 4/4 agents applied: 8–14 minutes. Full runner re-registration after recreate: Apple-silicon hosts ~2–4 minutes; emulated minis 20–30 minutes (a boot storm of ~15 Python processes under qemu — §4.4). The platform's liveness machinery (reaper, self-healing registration) absorbs the storm without operator attention.
- **Steady-state health:** at window close, ~69 live runners, zero stale registrations, zero dead-lettered production tasks attributable to fleet operations.

The headline is not that nothing went wrong — §4 is long — but that the *blast radius of each incident was one host or less*, and production work routed around every one of them, because no runner is special.

4. The incident catalog

Each entry: signature → root cause → recovery → permanent mitigation.

4.1 Sleep/wake and the zombie runners

Signature. Fleet status reports 33 live runners; 50+ containers are demonstrably working. Repair tooling treats the invisible runners' tasks as orphaned — a double-execution hazard. **Cause.** A desktop slept (or an emulated host stalled long enough to miss heartbeats). The orphan reaper — correctly — marked its runner records dead; a pruner then deleted them. On wake, runners resumed heartbeating *into the void*: the heartbeat wrote only a timestamp and could neither resurrect a record marked dead nor recreate a pruned one. Working, invisible, forever. **Recovery.** Container restart (re-registration on boot). **Mitigation (permanent).** The heartbeat became self-healing: it now performs an atomic *live-record-only* update and, on miss, re-registers the full server record — quarantine states respected, graceful shutdown untouched. Consumer hosts sleep; the registry now forgives them. This single change moved fleet-status truthfulness from "restart things until counts look right" to unattended.

4.2 DHCP drift and identity

Signature. After a router event, hosts resolve each other to stale addresses; agents pull from a registry that is no longer there. **Cause.** Address-based configuration on a network where addresses are leased. **Mitigation.** The stable-name server catalog (names + aliases resolved at startup/reconcile) replaced every hard-coded address, including the image registry's (`server3.local:5050`). Since adoption, drift is a non-event — the class is closed rather than handled.

4.3 Image-layer disk exhaustion (and the 113 GB mini)

Signature. One host's reconcile loops with `ENOSPC`; its Docker daemon becomes unresponsive; `docker` CLI hangs. **Cause.** Nine rollouts $\times$ ~ 2 GB image, and nothing ever deleted superseded tags. Six generations accumulated inside a Docker Desktop VM file that only grows, on the fleet's smallest disk. An unattended-operations blind spot: data-plane storage was carefully externalized (object store), while the *image* plane silently accreted per host. **Recovery.** The documented nuclear option, executed calmly: quit Docker, delete the VM file (`Docker.raw`), restart — everything inside was rebuildable from the registry; the agent re-pulled and recreated all runners. Twenty minutes, no data loss by construction. **Mitigation (permanent).** The fleet-agent now garbage-collects after every successful reconcile: keep the applied tag plus one rollback tag, delete the rest, prune dangling layers. Best-effort, never fails the reconcile. Image storage is now bounded at ~ 4 GB/host regardless of rollout cadence.

4.4 Emulation boot storms

Signature. After a recreate, a mini reports 1 of 15 runners for many minutes; naive monitoring reads it as an outage. **Cause.** Fifteen CPython processes cold-starting simultaneously under x86-on-ARM emulation, at load averages near 13. Not a failure — a boot envelope. **Mitigation.** Procedural, not code: rollout tooling and operator expectations treat 20–30 minutes of mini re-registration as normal; liveness machinery already tolerates it. The deeper lesson: on heterogeneous fleets, *time-to-registered is a per-host-class distribution*, and health checks must be parameterized by it.

4.5 Wedged container runtimes and stuck agents

Signature. One host's agent stops applying config versions; or a single runner exits repeatedly with code 124. **Cause class.** Docker Desktop VM corruption after disk pressure; transient daemon wedges; one recurring per-host container pathology. **Recovery.** `launchctl kickstart` for agents; VM rebuild for corruption (§4.3); per-container recreate otherwise. All are one-liners because no recovery ever involves preserving runner-local state — there is none.

4.6 What never happened

Classes conspicuous by absence, attributable to design: no split-brain (no leader to split), no queue corruption (single-writer tasks, atomic claims), no lost outputs from host failure (object-store durability), no cross-domain starvation from one flooded queue (derived task-list isolation), and no incident whose blast radius exceeded one host.

5. What earned its keep

Ranked by realized value in this environment:

1. **Runner disposability as the invariant.** Every recovery in §4 reduces to "delete and recreate," legal only because runners hold no authority — not even over their own identity. The test of the property is that the *most destructive* recovery (VM deletion) is also the safest.
2. **Authority in the data plane.** Config, claims, routing keys, outputs, even the applied-version record live in MongoDB/MinIO. The fleet has no privileged process whose loss matters except the infra host itself — a consciously accepted single point, mitigated by being the one machine treated as a pet.
3. **Self-healing registration** (§4.1) — the difference between a fleet that tolerates consumer power management and one that merely survives it.
4. **Derived routing** — task lists from namespaces, placement gates from host capability tiers. Where we deviated into configuration (a glob-topic filter passed literally into an exact-match claim), we got a silent weeks-long desync in which dedicated runners claimed nothing and catch-all runners quietly did their work; the incident is documented as the exception that argues for the rule.
5. **Reconciliation over orchestration.** The agent's package-manager shape (poll a version, converge, record) proved exactly powerful enough: nine rollouts, one command each. What it lacks against Kubernetes — surge/drain semantics, health-gated progressive rollout — cost us nothing at four hosts, and the drain machinery exists at the task layer instead (graceful reclaim on lease expiry), which is where a queue system actually needs it.

6. Limits, honestly

- **The infra host is a pet.** MongoDB, MinIO, and the registry share one machine with no failover. This is the model's stated boundary: the *runner* tier is informal; the coordination tier is small but must be cared for. Teams needing coordination-tier HA should rent it (managed Mongo/S3) and keep the informal runner tier.
- **Scale ceiling untested beyond the small.** Four hosts, ~70 runners, ~50k-step workflows. The claim-throughput math (atomic updates on indexed queries) suggests headroom of one to two orders of magnitude; we report only what we ran.
- **Security posture is LAN-trust.** Encrypted secrets exist for credentials, but the model assumes machines on the fleet network are friendly. This matches the small-team deployment; it is not multi-tenant.
- **One operator, high context.** Recoveries were fast partly because the operator (human + AI pair, per the thesis's authorship addendum) carried full-system context. The mitigations in §4 each convert one class of context-dependent recovery into an unattended behavior, which is the correct long-run direction; the catalog measures how far that conversion has progressed (three classes closed, two proceduralized).

7. Conclusion

A leaderless, data-plane-authoritative workflow runtime turns a pile of consumer machines into a legitimate execution tier. The incidents consumer hardware adds — sleep, drift, disk, emulation — are real but *shallow*: each reduced to a bounded recovery, and the recurring ones were closed permanently with small mechanisms (self-healing registration, image GC, name-based identity) rather than operational vigilance. The pattern we commend to small teams is exactly this shape: rent or nurture one stable coordination host; make every other machine disposable; derive rather than configure; and let the desks contribute their idle cores.

Artifacts: fleet tooling `scripts/lib/fleet/`, `fw fleet` command surface; self-healing heartbeat, image GC, and adaptive-poll commits; incident and rollout record in `docs/architecture/lessons-learned.md` and project memory; the operational envelope figures from the July 2026 v83→v91 rollout series.